

ADVANCED PROBLEM PROGRAMMING

SLO 2: Language Classification

1. Classify Java as Compiled/Interpreted and Explain Why

Java is generally classified as a **both compiled and interpreted language**.

Java uses a combination of **compilation** and **interpretation** to execute programs.

Step 1: Compilation

The Java source program is written in a file with the .java extension.

Example:

```
class Hello {  
    public static void main(String[] args) {  
        System.out.println("Hello World");  
    }  
}
```

The Java compiler javac converts the source code into **bytecode**.

Hello.java

↓

javac compiler

↓

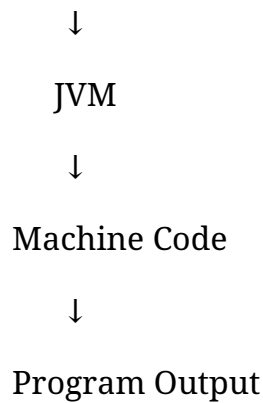
Hello.class

The .class file contains Java **bytecode**.

Step 2: Interpretation/Execution

The **Java Virtual Machine (JVM)** executes the bytecode.

Java Bytecode



The JVM can interpret bytecode and can also use **Just-In-Time (JIT) compilation** to compile frequently executed bytecode into native machine code.

Why is Java both compiled and interpreted?

- Java source code is **compiled** into bytecode by javac.
- The bytecode is executed by the **JVM**.
- The JVM may interpret the bytecode or compile it using the **JIT compiler**.

Therefore, Java is commonly described as a **compiled and interpreted language**.

2. Write a Simple Java Program and Explain How It Is Converted to Bytecode

Java Program

```
class Hello {  
  
    public static void main(String[] args) {  
  
        int a = 10;  
  
        int b = 20;  
  
        int sum = a + b;  
  
  
        System.out.println("Sum = " + sum);  
    }  
}
```

```
}  
}
```

Step 1: Create the Source File

Save the program as:

Hello.java

Step 2: Compile the Program

Use the Java compiler:

```
javac Hello.java
```

The compiler checks the Java program for syntax errors and converts it into bytecode.

The result is:

Hello.class

Step 3: Bytecode

The .class file contains **Java bytecode**.

Bytecode is an intermediate code that is designed to run on the JVM rather than directly on a specific computer's processor.

Step 4: Execute the Bytecode

Run the program using:

```
java Hello
```

The JVM loads and executes the bytecode.

Conversion Process

Hello.java

↓

Java Compiler (javac)

↓

Hello.class

↓

Java Bytecode

↓

JVM

↓

Machine Code

↓

Output

Output

Sum = 30

Important Point

The same .class bytecode can run on different operating systems as long as they have a compatible JVM. This is the basis of Java's "**Write Once, Run Anywhere**" (WORA) concept.

3. Differentiate Between Low-Level and High-Level Languages with Examples

Programming languages can be broadly classified into **low-level languages** and **high-level languages**.

Low-Level Language

A low-level language is closer to the computer's hardware and machine instructions.

Examples:

- Machine Language
- Assembly Language

High-Level Language

A high-level language is closer to human language and is easier for programmers to understand and write.

Examples:

- Java
- Python
- C++
- C#
- JavaScript

Difference Between Low-Level and High-Level Languages

Feature	Low-Level Language	High-Level Language
Meaning	Close to hardware	Close to human language
Ease of learning	Difficult	Easier
Programming	Complex	Simpler
Hardware dependency	Usually high	Usually lower
Execution	Very fast and hardware-specific	Requires compiler/interpreter/runtime
Portability	Low	Generally high
Examples	Machine Language, Assembly	Java, Python, C++

Example of Low-Level Language

Assembly language:

MOV AX, 05

ADD AX, 03

This is closer to the computer's processor instructions.

Example of High-Level Language

Java:

```
int x = 5 + 3;
```

```
System.out.println(x);
```

This is easier for humans to understand.